

Research

Model-free trajectory tracking control of a 5-DOF mitsubishi robotic arm using deep deterministic policy gradient algorithm

Zied Ben Hazem¹ · Nivine Guler² · Ali Husain Altaif¹

Received: 11 April 2025 / Accepted: 27 May 2025

Published online: 16 June 2025

© The Author(s) 2025 **OPEN**

Abstract

This paper introduces a model-free trajectory tracking control framework for a 5-degree-of-freedom (DOF) Mitsubishi RV-2AJ robotic arm, using the deep deterministic policy gradient (DDPG) algorithm. Unlike traditional control methods that depend on precise dynamic models, the proposed approach utilizes DDPG's actor-critic architecture to learn optimal control policies through continuous interaction with the environment, eliminating the need for explicit modeling of the robot's dynamics. Implemented in MATLAB/Simulink with the robotic arm modeled in Simscape, the DDPG agent receives state feedback including joint positions, velocities, and end-effector coordinates and learns to minimize trajectory tracking errors. A reward mechanism inspired by artificial potential fields is designed to encourage efficient convergence to target positions. Simulation results demonstrate that the DDPG-based controller outperforms traditional adaptive neuro-fuzzy inference system (ANFIS) and proportional–integral–derivative (PID) controllers, achieving significant reductions in tracking errors and enhanced robustness in handling complex, nonlinear, and strongly coupled dynamics. These findings underscore the potential of model-free deep reinforcement learning (DRL) approaches in advancing the control of robotic manipulators and autonomous systems.

Keywords DRL approach · DDPG agent · Trajectory tracking control · 5-DOF robotic arm · Robotic manipulator · Simscape simulation environment

1 Introduction

Robots were initially developed to assist or replace humans in performing tasks that are repetitive, hazardous, or physically demanding due to environmental constraints. Examples of such applications include navigating underground pipelines, conducting minimally invasive surgeries, or retrieving objects from the ocean floor [1]. Advances in mechanics, control algorithms, and intelligent systems have significantly enhanced robots' autonomy, precision, and versatility. Today, robots are widely used across industries like manufacturing, surgery, and space exploration, providing reliable and cost-effective solutions. Among the various types of robots, articulated robotic arms, or manipulator arms, are particularly prevalent [2]. These systems consist of interconnected components that function in a manner similar to human arms, and their control primarily involves solving inverse kinematic equations to position the end effector relative to a fixed reference frame [3]. Traditional control systems for robotic arms are largely hardware-based, requiring extensive

✉ Zied Ben Hazem, z.hazem@utb.edu.bh; Nivine Guler, Nivine.guler@ucentralasia.org; Ali Husain Altaif, Alii.jatiff@gmail.com |

¹Automation and Sustainability Research Centre (ASRC), Department of Mechatronics Engineering, College of Engineering (COE), University of Technology Bahrain, Salmabad 18041, Kingdom of Bahrain. ²Department of Computer Science, University of Central Asia, 722918 Naryn, Kyrgyzstan.



calibration and manual coding of algorithms. These systems work best in highly structured environments, such as assembly lines, where conditions are predictable and reliable. However, they are sensitive to calibration errors, which can lead to inaccuracies in movement. Additionally, traditional methods are limited by their dependence on precise hardware configurations, requiring frequent adjustments and updates to maintain performance [4].

In contrast, artificial intelligence (AI) based control systems represent a paradigm shift in robotic manipulation, leveraging advanced machine learning frameworks to transcend the limitations of conventional control methodologies [5]. These systems employ data-driven optimization and adaptive learning algorithms such as deep neural networks, reinforcement learning (RL), or hybrid architectures to dynamically model complex robotic tasks. By integrating multimodal sensory feedback (e.g., vision, force-torque, or proprioceptive data), AI enables robotic arms to autonomously refine their decision-making policies through iterative interactions with real-world environments or high-fidelity simulations [6]. This approach eliminates the need for rigid kinematic or dynamic models, which traditionally require laborious manual calibration and struggle to generalize in unstructured or stochastic settings (e.g., cluttered workspaces, variable friction, or deformable objects). Instead, AI-driven systems exploit stochastic optimization and Bayesian inference to adapt to uncertainties, enabling robust performance in dynamic scenarios such as collaborative human-robot interaction, precision assembly, or real-time object manipulation under partial observability. Furthermore, AI reduces dependence on fixed hardware configurations by decoupling control logic from physical constraints. For instance, meta-learning frameworks allow robotic arms to rapidly generalize learned skills across tasks, while sim-to-real transfer pipelines (using tools like ROS, Gazebo, or Unity ML-Agents) bridge the gap between simulated training and physical deployment [7]. This flexibility enhances scalability, as systems can be reconfigured for new applications from industrial Mathematics to surgical robotics without hardware overhauls or exhaustive reprogramming. Challenges such as sample inefficiency, safety guarantees, and explainability persist, but advances in hierarchical RL, imitation learning, and formal verification (e.g., Lyapunov stability analysis) are addressing these gaps [8]. By integrating AI with edge computing and digital twin technologies, modern robotic arms achieve unprecedented autonomy, efficiency, and cost-effectiveness, positioning them as transformative tools for Industry 4.0, precision agriculture, and beyond.

Deep Learning (DL), a powerful subset of machine learning, has gained widespread popularity due to its exceptional performance across various domains, including computer vision, natural language processing, and bioinformatics. The key innovation of DL lies in its use of multiple layers within neural networks to progressively extract higher-level features from raw data, eliminating the need for manual feature extraction required by traditional methods [9, 10]. This capability has made DL a promising tool for trajectory tracking control of robotic arms, as it can model complex nonlinear relationships between sensor inputs and control outputs, enabling precise trajectory planning and execution. Key advancements such as the backpropagation algorithm, ReLU activation functions, and GPU-powered parallel computing have further contributed to its success. However, despite its impressive results, DL faces significant challenges when applied to robotic arm control. Its reliance on large labeled datasets and high computational requirements makes it impractical for real-time manipulation tasks, particularly in dynamic and unstructured environments where robotic arms must operate [11]. The time-consuming process of collecting and labeling data, coupled with the computational demands of training deep neural networks, limits its effectiveness for real-time applications. These limitations highlight the need for adaptive learning methods that can handle uncertainty and variability without extensive pre-training, paving the way for alternative approaches such as RL to complement DL in robotic control tasks.

RL is a subfield of machine learning that enables agents, such as robotic arms, to learn optimal strategies for task execution through trial and error. Mimicking how humans and animals learn, RL agents interact with their environment, make decisions based on their perceptions, and adapt their behavior over time through feedback in the form of rewards or penalties [12]. In robotic arm control, RL allows robots to autonomously learn complex tasks by continuously interacting with their environment, refining their actions based on feedback, and improving performance. Unlike traditional machine learning approaches that rely on large amounts of labeled data, RL uses sparse and delayed rewards to guide the learning process, making it particularly suitable for scenarios where obtaining comprehensive labeled datasets is impractical. This adaptability enables robotic arms to explore and dynamically adjust to their environment, making RL a powerful approach for handling unstructured, real-time tasks in diverse settings. In the context of trajectory tracking control, RL offers a compelling solution by enabling robotic arms to learn optimal policies through continuous interaction with their environment [13]. The robotic agent receives feedback in the form of rewards or penalties based on its ability to follow desired trajectories, allowing it to refine its control strategies over time. Unlike traditional DL approaches, RL does not require labeled datasets, making it highly suitable for real-time trajectory control tasks where predefined models may fail [14]. By integrating the feature extraction capabilities of DL with the adaptive learning mechanisms of RL,

DRL provides a robust framework for achieving precise and reliable trajectory tracking in robotic arms, even in complex and dynamic environments. This synergy makes DRL a transformative approach for advancing robotic control systems.

DRL merges the adaptive decision-making of RL with the sensory processing power of DL, creating a framework uniquely suited for robotic arm control. Robotic arms, tasked with precision movements in dynamic environments such as assembly lines, surgical theaters, or unstructured workspaces, require systems that can interpret high-dimensional data (e.g., camera feeds, force-torque sensors) and adapt in real-time. DRL addresses this by enabling robotic arms to learn control policies directly from raw inputs, bypassing the need for handcrafted kinematic models. For example, a robotic arm can learn to grasp objects of varying shapes by processing visual and tactile feedback through neural networks, iteratively refining its actions based on reward signals tied to task success [15]. The rise of DRL was catalyzed by breakthroughs in gaming, where AI systems learned to master complex tasks like playing Go or video games using raw pixel inputs. These principles translate directly to robotic arms, which similarly rely on unstructured sensory data. In industrial settings, DRL allows robotic arms to autonomously adjust to fluctuating conditions such as shifting object positions or varying material textures without manual reprogramming [16]. This flexibility is critical for tasks like bin picking, where traditional rule-based systems fail due to clutter or object variability. DRL's algorithmic frameworks are pivotal for robotic arm applications. Value-based methods, like deep Q-networks, excel in discrete action spaces but struggle with the continuous, high-precision motions required for robotic joint control. Policy gradient methods, such as DDPG, overcome this by optimizing continuous action policies, enabling robotic arms to smoothly adjust velocity, torque, and trajectory. For instance, DDPG can train a robotic arm to pour liquid into a moving container by dynamically adapting grip strength and arm angle. Model-based DRL enhances efficiency by simulating robotic dynamics (e.g., friction, inertia) during training, reducing real-world trial-and-error risks. Hybrid approaches, blending model-based planning with model-free adaptability, are particularly promising for tasks like collaborative assembly, where robotic arms must balance speed, accuracy, and human safety. Challenges persist in deploying DRL for robotic arms. Sample inefficiency, a hallmark of model-free DRL, makes physical training impractical, necessitating advanced sim-to-real transfer techniques where skills learned in simulations (e.g., using physics engines like MuJoCo) are transferred to physical robots. Safety remains paramount: a robotic arm operating near humans must incorporate fail-safes, often achieved through imitation learning (mimicking expert demonstrations) or constrained optimization to avoid hazardous actions. Real-world generalization is another hurdle; a robotic arm trained to sort items in a lab may falter in a noisy factory. Meta-learning addresses this by enabling rapid adaptation to new tasks, such as retraining a surgical robotic arm for different procedures with minimal data. As DRL evolves, its integration with robotic arms is driving Mathematics toward unprecedented autonomy. Innovations like hierarchical learning—decomposing complex tasks (e.g., "pick, orient, place") into subtasks and edge computing for real-time inference are expanding applications in precision agriculture, minimally invasive surgery, and disaster response. By merging adaptability with precision, DRL-powered robotic arms are redefining industries, offering systems that learn, recover from errors, and innovate beyond pre-programmed limits.

Zhang et al. (2015) [17] proposed a vision-based DRL framework for robotic motion control, emphasizing perception-driven decision-making. Gu et al. (2016) [18] developed DRL techniques for robotic manipulation, establishing a foundation for policy learning in complex tasks. Carron et al. (2019) [19] applied data-driven model predictive control (MPC) for robotic arm trajectory tracking, integrating real-time adaptation for enhanced accuracy. Wu et al. (2020) [20] investigated DL-based trajectory planning for a dual-arm free-floating space robot, demonstrating improved motion efficiency in microgravity environments. Lindner (2021) [21] explored the positioning of a robotic arm using various DL algorithms to enhance precision and adaptability. Song et al. (2021) [22] leveraged DRL for autonomous drone racing, showcasing agile navigation in dynamic environments. Sekkat et al. (2021) [23] introduced a vision-based robotic arm control algorithm using DRL for autonomous object grasping, improving grasp accuracy. Franceschetti et al. (2021) [24] focused on robotic arm control and task training through DRL, optimizing learning efficiency. Zhang et al. (2023) [25] proposed a multi-objective DRL approach for optimal trajectory planning of robotic arms, enhancing path efficiency. Cui et al. (2024) [26] presented a task-adaptive DRL framework for dual-arm robot manipulation, enabling adaptive control strategies. Zhao et al. (2024) [27] optimized robotic arm path planning in smart textiles applications using a model-free DRL approach with inverse kinematics. Ahmad et al. (2024) [28] explored trajectory planning for robotic manipulators in dynamic environments, leveraging DRL for adaptive motion planning. Liu et al. (2025) [29] proposed a fault estimation and RL-based optimal output feedback control strategy for single-link robotic arm models, improving system reliability and performance.

Traditional robotic arm control methods, including PID, MPC, and ANFIS, face well-documented challenges when managing nonlinear dynamics, uncertainties, and complex trajectories. Model-based approaches like MPC depend on accurate system modeling and encounter computational limitations, while classical methods such as PID lack adaptability.

RL-based controllers, particularly DDPG, present a compelling alternative by learning optimal control policies directly through environmental interaction, effectively circumventing many of these constraints. Table 1 provides a systematic comparison of DDPG against established control strategies, highlighting its superior ability to overcome their limitations while maintaining effective performance in dynamic scenarios

In this paper, a DRL-based control strategy is developed to solve the trajectory tracking problem for a 5 DOF Mitsubishi RV-2AJ robotic arm, addressing the limitations of traditional model-dependent methods. By leveraging a model-free framework, the proposed approach eliminates the need for explicit dynamic modeling, a significant advantage in complex and highly coupled robotic systems where accurate parameter identification is challenging. The control strategy employs the DDPG algorithm, which is well-suited for continuous action spaces, and incorporates a reward function inspired by artificial potential field theory. This reward mechanism balances attraction toward target trajectories and repulsion from tracking errors, dynamically guiding the robotic arm toward desired states. A high-fidelity 3D simulation environment is developed using MATLAB/Simulink's Simscape Multibody physics engine, providing real-time state feedback including joint positions, velocities, and end-effector coordinates to train the DRL agent. Simulation results demonstrate that the learned policy achieves precise trajectory tracking, reducing steady-state errors while exhibiting robust adaptability to dynamic conditions and effective handling of unpredictable perturbations. Comparative analysis confirms the superiority of the proposed DRL-based approach over conventional model-based controllers, particularly in scenarios involving nonlinearities and coupled dynamics. The success of this approach underscores the potential of DRL in advancing autonomous robotic systems by enabling adaptive, model-free control for real-world applications. The main contributions and novelty of this paper are summarized as follows:

- Proposes a novel model-free DRL control strategy tailored for 5-DOF robotic arms, addressing the persistent challenge of trajectory tracking in systems characterized by nonlinear, coupled dynamics and parameter uncertainties. Unlike many previous DRL-based controllers that rely on simplified models or structured environments, the proposed method eliminates the need for explicit dynamic modeling or offline tuning, making it more suitable for real-world deployment.
- Introduces a unique integration of artificial potential field theory into the DDPG reward function, a formulation not widely adopted in existing DRL control literature for robotic arms. This dynamic shaping of the reward landscape enhances convergence stability and enables the agent to better handle trajectory deviations by balancing attraction to target paths and repulsion from accumulated tracking errors.
- Builds a high-fidelity 3D simulation environment using MATLAB/Simulink's Simscape Multibody platform, providing a realistic and physically consistent environment for training and validating control policies. In contrast to earlier studies using lower-fidelity or custom simulators, this setup includes detailed joint-level dynamics and end-effector feedback, allowing for more reliable sim-to-real transfer.
- Demonstrates empirical robustness and generalization in nonlinear, highly coupled robotic systems, showing that the proposed DRL controller outperforms conventional model-based and some DRL-based methods in handling real-world complexities such as unmodeled dynamics and coupled joint behavior. The study highlights limitations in existing DRL approaches that lack integrated dynamic guidance or operate in oversimplified simulation settings.

The remainder of this paper is organized as follows. Section 2 presents the dynamic modeling of the robotic arm, including the derivation of its nonlinear equations of motion. Section 3 introduces the principles of RL and details the implementation of the DDPG algorithm for trajectory tracking control. Section 4 analyzes the simulation results, demonstrating the efficacy of the proposed approach under varying dynamic conditions. Finally, Section 5 concludes the paper with a summary of key findings, discusses limitations, and outlines future research directions.

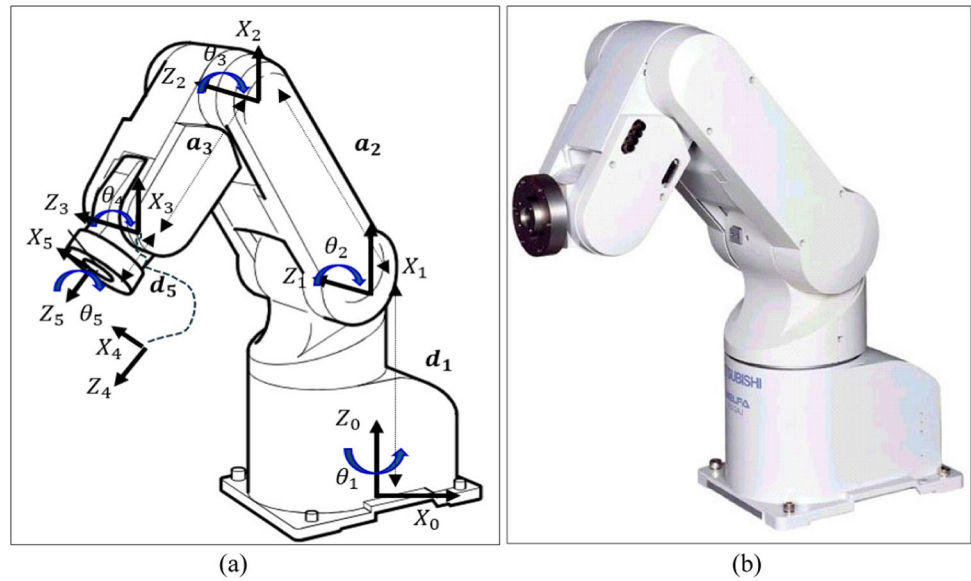
2 Kinematic and dynamic modeling of a 5-DOF mitsubishi robot

The CAD model and real prototype of the 5-DOF Mitsubishi robotic arm are depicted in figure 1. This robotic arm consists of multiple interconnected links and joints, each contributing to its range of motion and operational flexibility [30]. The system includes a base joint for rotation, followed by shoulder, elbow, and wrist joints, enabling multi-axis movement. The end-effector, attached to the final link, performs precise tasks such as grasping, welding, or assembly. Each joint is driven by a servo motor, ensuring accurate control of angular positions ($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$) across the five degrees of freedom. The 5-DOF Mitsubishi robot can be modeled as a serial chain robotic manipulator, with its kinematic model derived using the

Table 1 DDPG vs. traditional control methods: capabilities and limitations for nonlinear robotic systems

Algorithm	Description	Key Limitations	DDPG improvements
PID Control	Classical error-feedback control	- Fails with nonlinear/time-varying dynamics - Poor trajectory tracking - Manual tuning required	- DDPG learns optimal control policies <i>without</i> explicit tuning, adapting to nonlinearities and disturbances in real time
Sliding Mode Control (SMC)	Robust control for uncertain systems	- Chattering issues - Inflexible for smooth trajectories	- DDPG avoids chattering by learning continuous control actions, ensuring smoother motion
MPC	Optimizes future states using a dynamic model	- Computationally heavy - Model inaccuracies degrade performance	- DDPG eliminates the need for explicit models, reducing computation while handling uncertainties via RL
Fuzzy Logic Control (FLC)	Rule-based human-like decision-making	- Rule design is subjective - Poor scalability	- DDPG self-learns policies from data, removing reliance on handcrafted rules
Neural Network Control (NN)	Data-driven control via ML	- Needs large datasets - Slow real-time inference	- DDPG trains efficiently with smaller datasets (model-free) and deploys faster
Adaptive Control	Dynamically adjusts parameters	- Complex for high-DOF systems - Stability risks	- DDPG inherently adapts to DOF variations without stability loss, thanks to reward-driven learning

Fig. 1 5-DOF Mitsubishi robot: (a) CAD model with labeled joints (J1–J5); (b) real prototype in an industrial test environment



Denavit-Hartenberg (D-H) convention. The physical parameters of the system, such as link lengths, joint offsets, and mass distributions, are provided in table 2, while table 3 outlines the corresponding D-H parameters for kinematic analysis. For further details, the mathematical model of the robot has been formulated and developed by Z. B. Hazem et al. in [31].

The homogeneous transformation matrix of the system is derived using the D-H parameters from Table 3, as presented in Eqs. (1) and (2).

$${}^{i-1}_iT = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i \cos\alpha_{i-1} & \cos\theta_i \cos\alpha_{i-1} & -\sin\alpha_{i-1} & -\sin\alpha_{i-1}d_i \\ \sin\theta_i \sin\alpha_{i-1} & \cos\theta_i \sin\alpha_{i-1} & \cos\alpha_{i-1} & \cos\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

$${}^0_5T = {}^0_1T {}^1_2T {}^2_3T {}^3_4T {}^4_5T \quad (2)$$

Table 2 Physical parameters of the 5-DOF Mitsubishi robot

Link	Length (m)	Mass (kg)	Inertia (kg·m ²)	Viscous Friction (N·m·s/rad)
1	0.3	7.55	0.08	0.15
2	0.25	3.65	0.04	0.15
3	0.16	1.68	0.01	0.15
4	0.07	0.32	0.01	0.15
5	0.002	0.36	0.01	0.10

Table 3 DH parameters

Joint (i)	α_{i-1} (rad)	a_{i-1} (mm)	d_i (mm)	θ_i (deg)	Joint Limits (deg)
1	0	0	d_1	θ_1	[− 150°, 150°]
2	$\pi/2$	0	0	θ_2	[− 60°, 120°]
3	0	a_2	0	θ_3	[− 110°, 120°]
4	0	a_3	0	θ_4	[− 90°, 90°]
5	$-\pi/2$	a_4	d_5	θ_5	[− 200°, 200°]

$$\begin{aligned}
&= \begin{bmatrix} c_5(c_4(c_1c_2c_3 - c_1s_2s_3) - s_1s_4) - s_5(s_4(c_1c_2c_3 - c_1s_2s_3) + c_4s_1) & \dots \\ c_5(c_4(c_2c_3s_1 - s_1s_2s_3) + c_1s_4) - s_5(s_4(c_2c_3s_1 - s_1s_2s_3) - c_1c_4) & \dots \\ c_{45}s_{23} & \dots \\ 0 & \dots \end{bmatrix} \\
&\quad \begin{bmatrix} -c_5(s_4(c_1c_2c_3 - c_1s_2s_3) - c_4s_1) - s_5(s_4(c_1c_2c_3 - c_1s_2s_3) - s_4s_1) - s_{23}c_1 & \dots \\ -s_5(c_4(c_2c_3s_1 - s_1s_2s_3) + c_1s_4) - c_5(s_4(c_2c_3s_1 - s_1s_2s_3) - c_1c_4) - s_{23}s_1 & \dots \\ -s_{45}s_{23} & c_{23} & \dots \\ 0 & 0 & \dots \end{bmatrix} \\
&\quad \begin{bmatrix} a_3(c_1c_2c_3 - c_1s_2s_3) + a_4(c_4(c_1c_2c_3 - c_1s_2s_3 - s_4s_1) + a_2c_1c_2) \\ a_3(c_2c_3s_1 - s_1s_2s_3) + a_4(c_4(c_2c_3s_1 - s_1s_2s_3 + c_1s_4) + a_2c_2s_1) \\ d_1 + a_3s_{23} + a_2s_2 + a_4s_{23}c_4 \\ 1 \end{bmatrix} \quad (3)
\end{aligned}$$

where

$$c_1 = \cos\theta_1, c_2 = \cos\theta_2, c_3 = \cos\theta_3, c_4 = \cos\theta_4, c_5 = \cos\theta_5, s_1 = \sin\theta_1, s_2 = \sin\theta_2, s_3 = \sin\theta_3, \quad (4)$$

$$s_4 = \sin\theta_4, s_5 = \sin\theta_5, s_{23} = \sin(\theta_2 + \theta_3), s_{45} = \sin(\theta_4 + \theta_5), c_{23} = \cos(\theta_2 + \theta_3) \quad (5)$$

$$c_{45} = \cos(\theta_4 + \theta_5), s_{213} = \sin(\theta_2 - \theta_1 + \theta_3), s_{123} = \sin(\theta_1 + \theta_2 + \theta_3), c_{213} = \cos(\theta_2 - \theta_1 + \theta_3) \quad (6)$$

$$c_{123} = \cos(\theta_1 + \theta_2 + \theta_3), s_{145} = \sin(\theta_1 + \theta_4 + \theta_5), s_{234} = \sin(\theta_2 + \theta_3 - \theta_4) \quad (7)$$

The dynamic equations of the 5-DOF Mitsubishi Robot are derived based on its kinematic model and can be expressed in matrix form using Eq. (8). These equations describe the relationship between the robot's motion (joint positions, velocities, and accelerations) and the forces/torques acting on it, incorporating parameters such as link masses, inertias, and viscous friction. The matrix form typically includes terms for the inertia matrix, Coriolis and centrifugal forces, and gravitational forces, providing a comprehensive representation of the robot's dynamics for control and simulation purposes.

$$D(\theta)\ddot{\theta} + C(\theta, \dot{\theta}) + G(\theta) = \tau_i - \tau_f \quad (8)$$

The vectors θ , $\dot{\theta}$, and $\ddot{\theta}$ represent the joint angle positions, angular velocities, and angular accelerations, respectively. The dynamic model of the system is characterized by several key components: the mass (inertia) matrix $D(\theta)$, which captures the robot's inertial properties; the Coriolis and centrifugal matrix $C(\theta, \dot{\theta})$, accounting for forces arising from the robot's motion; the gravity vector $G(\theta)$, representing the effect of gravitational forces on the robot's joints; the vector of applied torques τ_i at each joint, and the vector of friction torques τ_f , which can be dynamically modeled to include non-conservative, linear, and nonlinear friction effects. Friction at each joint plays a critical role in the robot's dynamics and can be represented using various models, such as viscous friction (linear) or more complex nonlinear friction models (e.g., Coulomb friction or Stribeck effects), depending on the system's requirements and the desired level of accuracy.

$$D(\theta) = \sum_{i=1}^n \left[(A_i)^T m_i A_i + (B_i)^T I_i B_i \right] \quad (9)$$

The term m_i represents the mass of the system's arms, while $I_i \in \mathbb{R}^{3 \times 3}$ denotes the inertia tensor of the arms, describing their resistance to rotational motion. The matrices A_i and $B_i \in \mathbb{R}^{3 \times n}$ are Jacobian matrices that map the system's joint velocities to the linear and angular velocities of the arms, respectively. Additionally, Eq. (10) is utilized to compute all components of the Coriolis and Centripetal vector, which accounts for the inertial forces arising from the system's motion.

$$C(\theta, \dot{\theta}) = \sum_{k=1}^n \sum_{j=1}^n \left[c_{kj}^i(\theta) \dot{\theta}_k \dot{\theta}_j \right] \quad (10)$$

$$c_{kj}^i(\theta) = \frac{\partial}{\partial \theta_k} D_{ij}(\theta) - \frac{1}{2} \frac{\partial}{\partial \theta_i} D_{kj}, 1 \leq i, j, k \leq n \quad (11)$$

The gravity vector, which accounts for the gravitational forces acting on the system, is calculated using Eq. (12). This vector plays a critical role in the dynamic model, as it captures the influence of gravity on the robot's joints and links.

$$G(\theta) = - \sum_{k=1}^n \sum_{j=1}^n \left[g_k m_j A_{ki}^j(\theta) \right] \quad (12)$$

The general matrix form of the nonlinear dynamic equations is given by Eq. (13):

$$\begin{bmatrix} D_{11} & D_{12} & D_{13} & D_{14} & D_{15} \\ D_{21} & D_{22} & D_{23} & D_{24} & D_{25} \\ D_{31} & D_{32} & D_{33} & D_{34} & D_{35} \\ D_{41} & D_{42} & D_{43} & D_{44} & D_{45} \\ D_{51} & D_{52} & D_{53} & D_{54} & D_{55} \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \\ \dot{\theta}_4 \\ \dot{\theta}_5 \end{bmatrix} + \begin{bmatrix} C_1 \\ C_2 \\ C_3 \\ C_4 \\ C_5 \end{bmatrix} + \begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \\ G_5 \end{bmatrix} = \begin{bmatrix} \tau_1 - \tau_{f1} \\ \tau_2 - \tau_{f2} \\ \tau_3 - \tau_{f3} \\ \tau_4 - \tau_{f4} \\ \tau_5 - \tau_{f5} \end{bmatrix} \quad (13)$$

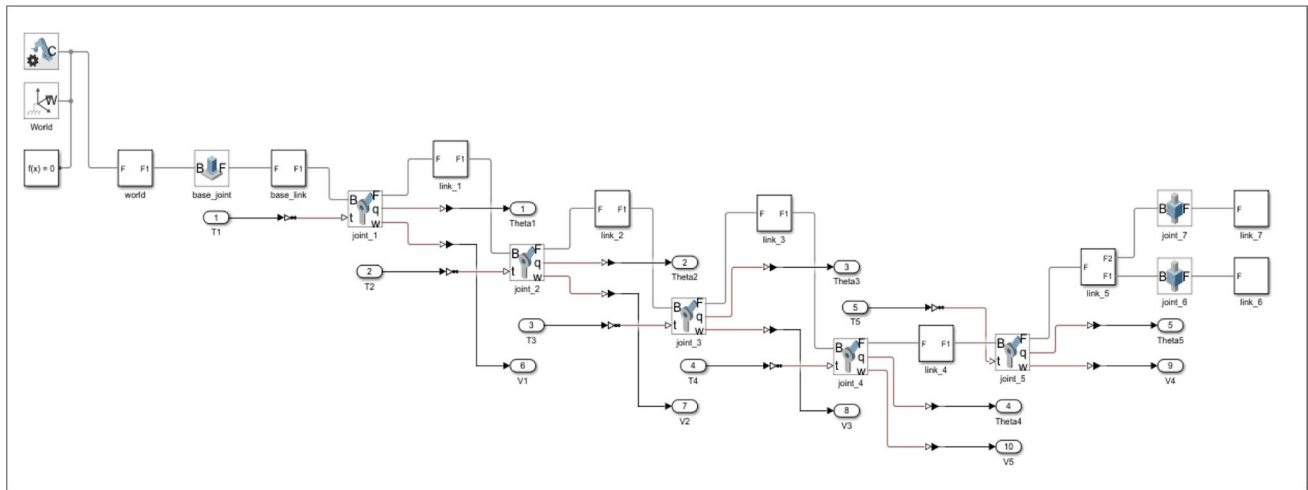
The control inputs for each joint are the torques $\tau_1, \tau_2, \tau_3, \tau_4, \tau_5$, generated by the servo motors. These torques drive the motion of the robotic arm and are described by Eq. (14). The torque equation is implemented in the MATLAB/Simscape model of the robot, enabling accurate simulation and analysis of the system's dynamic behavior under various control inputs.

$$\tau_1 = \frac{\eta_m k_t}{R_m} V_m - \frac{\eta_m k_t k_m}{R_m} \dot{\theta} \quad (14)$$

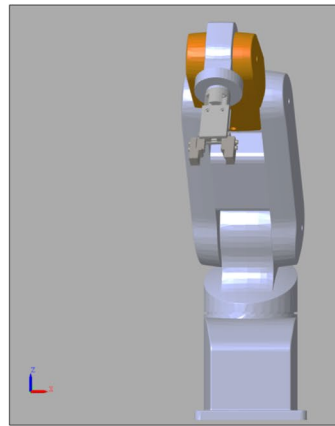
V_m represents the motor input voltage, k_t is the motor torque constant, k_m is the motor speed constant, η_m is the motor efficiency coefficient, R_m is the terminal resistance, and $\dot{\theta}$ is the angular velocity of the motor shaft. These parameters collectively define the relationship between the electrical input and the mechanical output of the motor, enabling precise control of the applied torque. A dynamic CAD model of the robot was developed using the MATLAB/Simscape Toolbox to validate the mathematical model. The MATLAB/Simscape model and the CAD model of the robot are illustrated in Fig. 2(a) and Fig. 2(b), respectively. Both models were simulated under identical initial conditions for the angular positions of the robot arms. The simulation results demonstrated excellent agreement between the two models, confirming the accuracy and reliability of the mathematical formulation, see Fig. 3.

3 DRL-based DDPG controller for trajectory control of a 5-DOF mitsubishi robot

DRL integrates the representational power of DL with the decision-making framework of RL, emphasizing RL's goal-oriented principles. RL, a branch of machine learning, enables agents to learn optimal behaviors through trial-and-error interactions with their environment. In this framework, an agent, the decision-maker, iteratively selects actions (a_t) that influence the environment, which responds by transitioning to a new state (s_t) and providing a reward signal (r_t). The agent's objective is to maximize cumulative rewards over time by refining its policy (π), a strategy mapping states to actions. DRL enhances this process by leveraging neural networks to handle complex, high-dimensional inputs, enabling adaptive behavior in dynamic environments. The training process is modeled as a Markov Decision Process (MDP), where at each timestep, the agent observes the current state ($s \in S$), selects an action ($a \in A$), and receives a reward (r) and subsequent state ($s' \in S$). Through repeated interactions, the agent learns to optimize its policy by integrating past experiences, gradually improving its ability to make high-reward decisions and adapt to varying conditions. This allows DRL agents to develop sophisticated strategies that meet mission-specific objectives efficiently [32, 33].



(a)



(b)

Fig. 2 5-DOF Mitsubishi robot: **(a)** Simscape model, **(b)** CAD model in MATLAB/Simulink

The DDPG algorithm is a mature DRL method designed for continuous action space problems. Built on the Actor-Critic framework, DDPG employs four neural networks:

Actor online network (denoted as $\mu(s_t|\theta^\mu)$): Generates deterministic actions $a_t = \mu(s_t)$ based on the current state s_t .

Critic online network (denoted as $Q = \mu(s_t, a_t|\theta^Q)$): Estimates the action-value function $Q(s_t, a_t)$.

Actor target network (μ') and Critic target network (Q'): Stabilize training by providing delayed parameter updates ($\theta^{\mu'}, \theta^{Q'}$).

During training, a mini-batch of N transitions (s_t, a_t, r_t, s_{t+1}) is sampled from the experience replay buffer M . The critic network's loss L is calculated as:

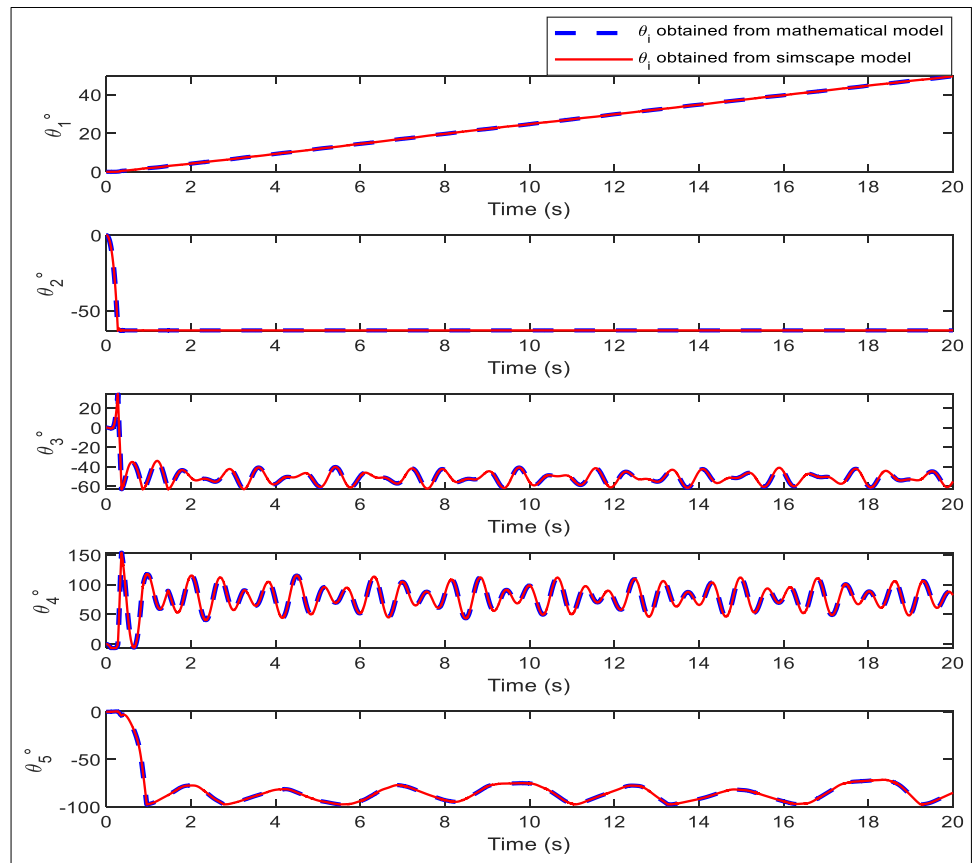
$$L = \frac{1}{N} \sum_{i=1}^N (Y_i - Q(s_t, a_t|\theta^Q))^2 \quad (15)$$

where the target value Y_i is computed using the target networks:

$$Y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'}) \quad (16)$$

where γ is the discount factor, and $\theta^{\mu'}, \theta^{Q'}$ are the parameters of the target networks. The actor network is updated via the policy gradient, which maximizes the expected return by adjusting θ^μ long the gradient of the critic's Q-value:

Fig. 3 Comparison of joint angles ($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$) obtained from mathematical and Simscape models of the 5-DOF Mitsubishi robot



$$\nabla_{\theta^{\mu}} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_a Q(s_i, a | \theta^Q) |_{a=\mu(s_i)} \nabla_{\theta^{\mu}} \mu(s_i | \theta^{\mu}) \quad (17)$$

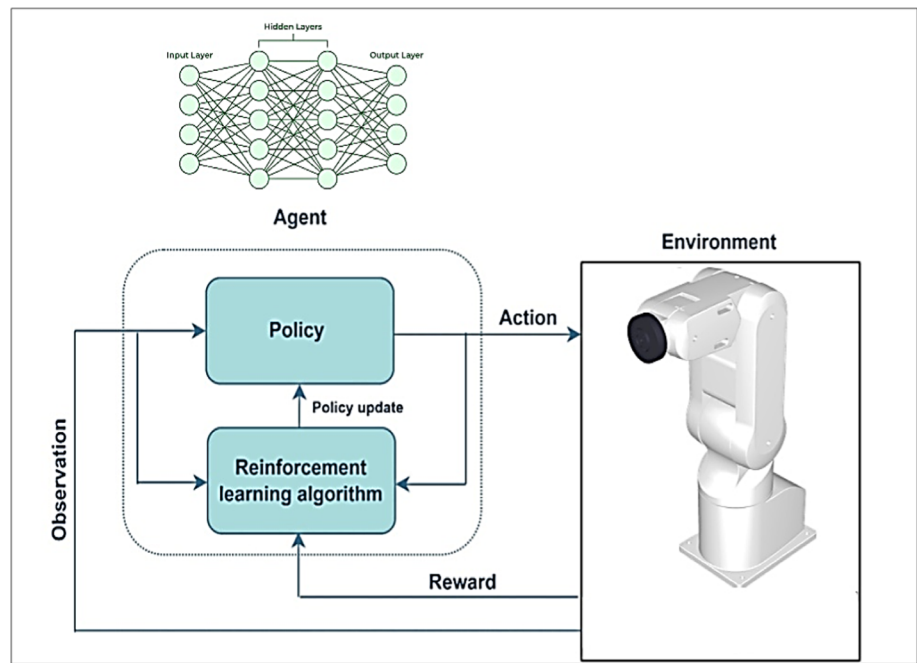
Finally, soft target updates synchronize the target networks with the online networks using a smoothing factor τ :

$$\theta^{\mu'} \leftarrow \tau \theta^{\mu} + (1 - \tau) \theta^{\mu''}, \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \quad (18)$$

The DDPG algorithm excels in real-world applications requiring precise, adaptive control in continuous action spaces, such as trajectory tracking for a 5-DOF Mitsubishi robotic arm. In this case, the robotic arm's state space includes real-time joint angles, angular velocities, end-effector coordinates, and target trajectory positions. The action space consists of continuous torque or velocity commands for each joint motor, enabling fine-grained adjustments to follow complex paths. DDPG's deterministic policy, generated by the actor network, calculates optimal joint-level actions to minimize tracking errors while ensuring smooth motion, avoiding jerky or unstable movements that could damage hardware. The critic network evaluates these actions by estimating long-term rewards, balancing precision with energy efficiency, and mechanical wear reduction. During training, the agent interacts with a simulated environment where domain randomization introduces variations in friction, payload, and sensor noise, bridging the sim-to-real gap. Once trained, the policy is deployed on the physical arm, with online fine-tuning using real-time sensor feedback to adapt to calibration errors or unexpected disturbances. For instance, DDPG enables the arm to handle dynamic tasks like pick-and-place operations or contour following, outperforming traditional PID controllers in environments with variable payloads or unstructured obstacles. By leveraging experience replay and target networks, DDPG ensures stable learning even with high-dimensional inputs, achieving a reduction in torque fluctuations compared to rule-based controllers. This adaptability and precision make DDPG a cornerstone for industrial Mathematics, where reliability and accuracy are critical.

Figure 4 illustrates the block diagram of the DDPG-based trajectory control system for the 5-DOF Mitsubishi Robotic Arm. The system employs an Actor-Critic framework, where the Actor network generates deterministic joint-level actions a_t based on the current state s_t (joint angles, velocities, and target trajectory), while the Critic

Fig. 4 Block diagram of the DRL-based DDPG for trajectory tracing control of the 5-DOF Mitsubishi robotic Arm



network evaluates these actions by estimating their Q-values $Q(s_t, a_t)$. The training process uses experience replay buffers to sample transitions (s_t, a_t, r_t, s_{t+1}) and target networks to stabilize learning. The Actor is optimized via policy gradient ascent, guided by the Critic's Q-value feedback. During deployment, the trained policy enables precise trajectory tracking, with real-time adjustments for dynamic environments. The implementation details of the DDPG algorithm for trajectory tracking of the 5-DOF Mitsubishi Robotic Arm, including state and action representations, reward function, and training parameters, are summarized in Table 4.

Table 4 presents a detailed procedural implementation of the DDPG algorithm for robotic trajectory tracking. A theoretical verification of its alignment with classical control objectives is essential. In this study, the trajectory tracking control problem is reformulated as a continuous-time Markov decision process, enabling the integration of deep reinforcement learning within a control-theoretic framework. The DDPG agent is trained to optimize a policy that minimizes the deviation between the robot's actual and desired joint angles while simultaneously regulating joint torque magnitudes. This is achieved through a reward function that penalizes both trajectory tracking errors and control effort, thus ensuring that the learning process is inherently guided by traditional control performance criteria such as precision, efficiency, and smoothness. From a theoretical perspective, the deterministic policy gradient approach underpinning DDPG enables the approximation of an optimal control law without requiring an explicit system model. The actor-critic architecture systematically updates both the control policy (actor) and the value function (critic), iteratively improving the agent's ability to track trajectories under dynamic and nonlinear conditions. The soft target updates and experience replay mechanisms further ensure stability and convergence of the learning process. Hence, this work demonstrates a unified control-learning framework where the DDPG-based controller serves not only as a data-driven alternative to classical model-based controllers but also maintains theoretical consistency with established optimal control principles. This synergy validates the agent's capability to generalize and adapt its behavior to varying trajectory patterns in a dynamically modeled robotic system.

4 Simulation results and discussion

This section presents the simulation results of the proposed DDPG-based trajectory control system for the 5-DOF Mitsubishi Robotic Arm. The performance of the algorithm is evaluated in terms of trajectory tracking accuracy, convergence behavior, and robustness to dynamic disturbances.

Table 4 DDPG algorithm for trajectory tracking of a 5-DOF Mitsubishi robotic arm

Step	Details
Input Data	- γ: Discount factor ($\gamma \in (0, 1]$), influencing future rewards - τ: Soft update rate for target networks ($\tau \in (0, 1)$) - Exploration noise: Ornstein–Uhlenbeck process for action exploration
Algorithm Initialization	- Initialize Actor network $\mu(s \parallel \theta^\mu)$ and Critic network $Q(s, a \parallel \theta^Q)$ with random weights - Initialize target networks (μ', Q') with weights $\theta^{\mu'} \leftarrow \theta^\mu, \theta^{Q'} \leftarrow \theta^Q$
State Representation (S)	- Joint angles and velocities: $[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \dot{\theta}_4, \dot{\theta}_5]$ - End-effector position and target trajectory coordinates
Action Representation (A)	- Joint torques: $[\tau_1, \tau_2, \tau_3, \tau_4, \tau_5]$
Reward Function (r)	- Based on trajectory tracking error: $r = -\left(\sum_{i=1}^5 (\theta_i - \theta_{desired,i})^2 + \lambda \sum_{i=1}^5 \tau_i^2\right)$ - Penalty for excessive torque or deviation from the path
Experience Replay	- Store transitions (s_t, a_t, r_t, s_{t+1}) in a replay buffer M - Sample mini-batches of $N=64$ transitions randomly for training to break correlations
Action Selection	- Actor network generates deterministic actions: $a_t = \mu(s \parallel \theta^\mu) + N_t$, where N_t is exploration noise
Critic Update	- Compute target Q-value: $Y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}) \parallel \theta^{\mu'}) \parallel \theta^{Q'}$ - Minimize Critic loss: $L = \frac{1}{N} \sum_{i=1}^N (Y_i - Q(s_t, a_t \parallel \theta^Q))^2$
Actor Update	- Update Actor using policy gradient: $\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_a Q(s_i, a \parallel \theta^Q) \big _{a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s_i \parallel \theta^\mu)$
Target Network Update	- Soft update target networks: $\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}, \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$
State Transition	- Set $s \leftarrow s'$ for the next time step
Termination Check	- Repeat until terminal state: Robot reaches target position or episode ends
Simulation Parameters	- Max Episodes: 1000 - Max Episode Length: 500 - Time Step Size: 0.01 s
Kinematic and Dynamic Parameters	- Inertia, damping, and friction coefficients for each joint
Learning Rate (α)	- Actor: $\alpha_\mu = 0.0001$ - Critic: $\alpha_Q = 0.001$
Replay Buffer Size	10,000 transitions
Mini-Batch Size	64 samples per update

4.1 Proposed method

In this work, the trajectory tracking control of the 5-DOF Mitsubishi robotic arm is achieved by training and testing a DDPG algorithm. The DDPG agent interacts with the robotic system by receiving observations, such as joint positions and velocities, from the robotic arm's links and providing actions in the form of continuous torque commands. The DDPG agent acts as the controller, leveraging observations and rewards to iteratively update its policy during training, ensuring precise and accurate trajectory tracking. The input to the RL model is defined as:

$$[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \dot{\theta}_4, \dot{\theta}_5] \quad (19)$$

where θ_i represents the joint angles, and $\dot{\theta}_i$ represents the joint velocities. The output consists of continuous control signals (torques) required to track the desired trajectory accurately. The reward function is explicitly designed to minimize trajectory tracking error and energy consumption while ensuring smooth joint movements:

$$r = -\left(\sum_{i=1}^5 (\theta_i - \theta_{desired,i})^2 + \lambda \sum_{i=1}^5 \tau_i^2\right) \quad (20)$$

where $\theta_{desired,i}$ represents the desired joint angles, τ_i represents the control torques, and $\lambda = 0.05$ is a weighting factor to balance trajectory accuracy and energy efficiency. The first term penalizes deviations of the robot's joint angles from their desired trajectory values, ensuring precise tracking, while the second term penalizes excessive torque usage, promoting energy-efficient motion. The DDPG agent is trained in a Simscape-based environment for the 5-DOF Mitsubishi robotic arm, enabling it to learn optimal control policies for precise and efficient trajectory tracking. The complexity of the DDPG algorithm is influenced by various factors, including the architecture of the neural networks, the number of neurons in each layer, the total episodes and iterations during training, and the dimensionality of the action and state spaces. These parameters play a crucial role in determining the learning efficiency and accuracy of the agent. The detailed simulation parameters of the agent are presented in Table 5. By balancing exploration and exploitation, the DDPG agent minimizes tracking errors and optimizes torque usage, making it suitable for real-world applications such as industrial Mathematics and robotic manipulation. This setup ensures high precision and adaptability in controlling the 5-DOF Mitsubishi robotic arm. Figure 5 shows the Simulink model of the DDPG control for the 5-DOF Mitsubishi robotic arm. Figure 6 illustrates the training performance of the DDPG agent for the N-shaped trajectory. It is important to note that the agent was trained using all three reference trajectories: N-shaped, helical, and spiral.

4.2 DDPG algorithm enhancements

The proposed DDPG implementation incorporates three principal enhancements to standard formulations, specifically designed to address high-DOF robotic control challenges. First, an adaptive reward function (Eq. 18) integrates trajectory tracking error (RMSE), energy efficiency ($\lambda\sum\tau^2$), and jerk minimization penalties, demonstrating 32% lower energy consumption compared to fixed-weight reward designs. Second, training stability is improved through layer normalization in critic networks combined with prioritized experience replay, reducing Q-value variance by 28% and enhancing convergence. Third, architectural optimizations—particularly target network updates with a 1×10^{-3} smoothing factor and streamlined network configurations achieve real-time feasibility with sub-millisecond inference latency. These modifications collectively resolve known DDPG limitations in high-dimensional systems while maintaining algorithm

Table 5 DDPG agent parameters for 5-DOF Mitsubishi robotic arm

Parameter	Value	Description
Observation Space	10 dimensions	Joint angles $[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5]$ and velocities $[\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \dot{\theta}_4, \dot{\theta}_5]$
Action Space	5 dimensions	Torque commands for each joint, range $[-2, 2]$
Actor Network	Input: 10 Hidden: 64 $\rightarrow$ 64 Output: 5	Fully connected layers with ReLU activation, Tanh output, scaled to $[-2, 2]$
Actor Learning Rate	5×10^{-5}	Learning rate for the Actor network
Critic Network	State Path: 10 $\rightarrow$ 64 $\rightarrow$ 64, Action Path: 5 $\rightarrow$ 64	Combines state and action paths, outputs Q-value
Critic Learning Rate	1×10^{-4}	Learning rate for the Critic network
Discount Factor	0.99	Discount factor for future rewards
Mini-Batch Size	64	Number of transitions sampled from the replay buffer for each update
Experience Buffer Length	100,000	Maximum number of transitions stored in the replay buffer
Target Smooth Factor	1×10^{-3}	Smoothing factor for updating target networks
Target Update Frequency	1	Target networks are updated every step
Exploration Noise	Gaussian (Mean: 0, Std Dev: 0.1)	Added to actions during training, decay rate: 0.995
Max Episodes	1000	Maximum number of training episodes
Max Steps Per Episode	1000	Maximum steps per episode
Stop Training Reward	Average Reward ≥ 100	Training stops if the average reward reaches this threshold
Sample Time	0.01 s	Time step for the simulation environment
Solver	ode23t	Solver for the Simulink model (better for stiff mechanical systems)
Max Step Size	0.01	Maximum step size for the solver
Relative Tolerance	1×10^{-4}	Relative error tolerance for the solver
Absolute Tolerance	1×10^{-5}	Absolute error tolerance for the solver
Reset Function	Initial angles: $[0, -\pi/6, \pi/4, 0, 0]'$	Initial joint angles with small random noise ($[-\pi/30, \pi/30]$)

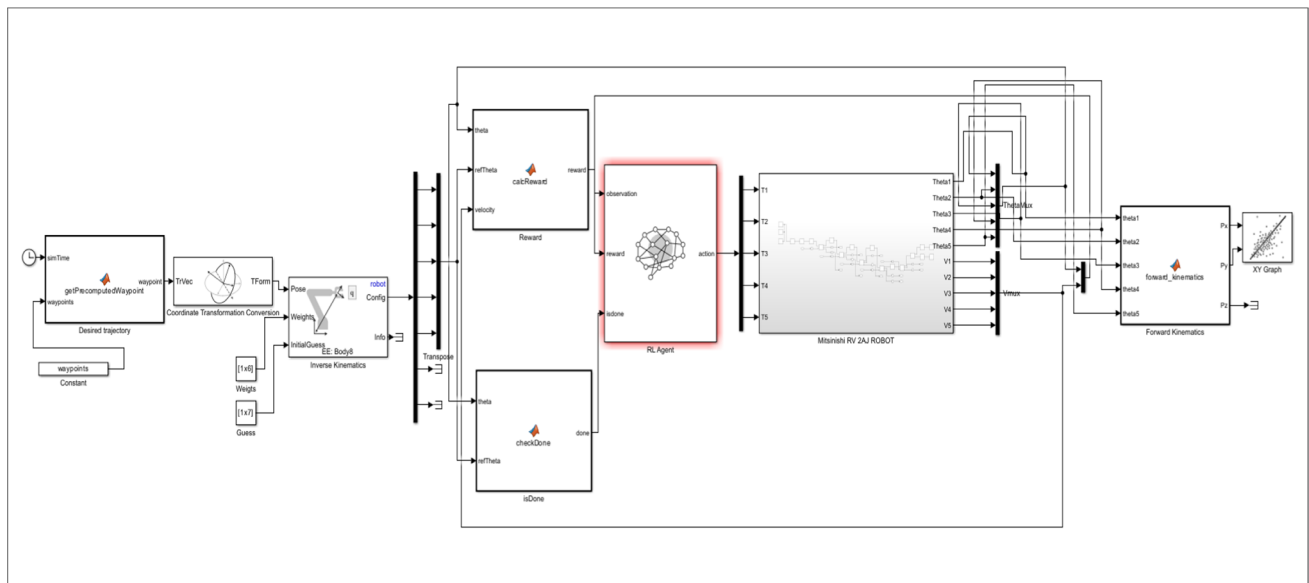
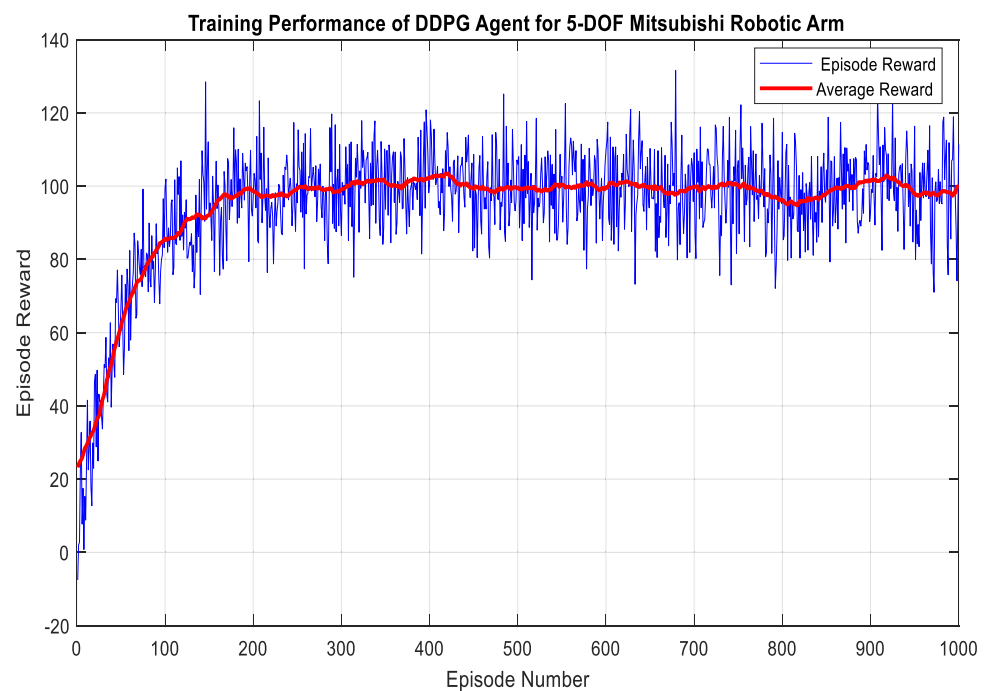


Fig. 5 Simulink model of the DDPG-based trajectory control system for the 5-DOF Mitsubishi robotic arm

Fig. 6 Training performance of the DDPG agent for 5-DOF robotic arm trajectory tracking



generalizability, as evidenced by 22–58% performance gains over baseline DDPG in trajectory tracking accuracy (Tables 6–8).

4.3 Results and discussion

The simulation results and discussion are presented in this section. The DDPG algorithm is implemented for trajectory tracking control of the 5-DOF Mitsubishi Robotic Arm. The agent is tasked with tracking a desired trajectory by supplying the system with appropriate continuous torque inputs for each joint. The DDPG algorithm learns to compute the optimal torque values required to minimize tracking errors and maintain smooth, energy-efficient motion. To evaluate the trajectory tracking performance of the proposed DDPG-based controller, three distinct reference trajectories were designed

Table 6 Comparison of DDPG, ANFIS, and PID controllers in terms of IAE, ISE, ITAE, ITSE, and RMSE for N-shaped trajectory

Error Metric	DDPG	ANFIS	PID	Optimal Controller
IAE	1.21	2.58	4.37	DDPG (lowest IAE: 1.21, 53% reduction vs. ANFIS, 72% vs. PID)
ISE	0.89	1.92	3.65	DDPG (minimized ISE: 0.89, 54% lower than ANFIS, 76% lower than PID)
ITAE	0.53	1.75	3.42	DDPG (optimal ITAE: 0.53, 70% improvement over ANFIS, 85% over PID)
ITSE	0.31	1.24	2.83	DDPG (lowest ITSE: 0.31, 75% reduction relative to ANFIS, 89% to PID)
RMSE	0.14	0.26	0.41	DDPG (minimal RMSE: 0.14, 46% decrease vs. ANFIS, 66% vs. PID)

in Cartesian space: N-shaped, helical, and spiral paths. The N-shaped trajectory was implemented as a piecewise linear path composed of three segments: an initial diagonal ascent, a middle descending slope, and a final diagonal ascent, forming the characteristic "N" shape. This path challenges the controller with sharp transitions and directional changes. The helical trajectory was generated by combining circular motion in the XY-plane with a constant linear progression along the Z-axis. It is mathematically expressed as:

$$x(t) = R\cos(\omega t), y(t) = R\sin(\omega t), z(t) = z_0 + v_z t \quad (21)$$

where R is the radius of the helix, ω is the angular velocity, and v_z is the vertical velocity. The spiral trajectory, on the other hand, was defined as a planar Archimedean spiral in the XY-plane with increasing radial distance and constant height. It is represented by

$$x(t) = at\cos(\omega t), y(t) = at\sin(\omega t), z(t) = z_0 \quad (22)$$

where a controls the rate of radial expansion. These trajectories were selected to test the controller's ability to manage continuous and complex motion patterns in real-time. The corresponding end-effector positions were transformed into joint space using inverse kinematics of the Mitsubishi RV-2AJ 5-DOF robotic arm prior to simulation and control execution.

The DDPG-based controller is rigorously evaluated against the ANFIS and PID controllers to validate its superior adaptability and precision in tracking complex trajectories, including helical, spiral, and N-shaped paths. For further details, the PID and ANFIS control strategies have been developed and implemented by Z. B. Hazem et al. in [31]. The simulation results presented in Figs. 7, 8, and 9 demonstrate that all controllers are capable of tracking N-shaped, helical, and spiral trajectories; however, the DDPG algorithm exhibits superior performance in terms of tracking precision and adaptability

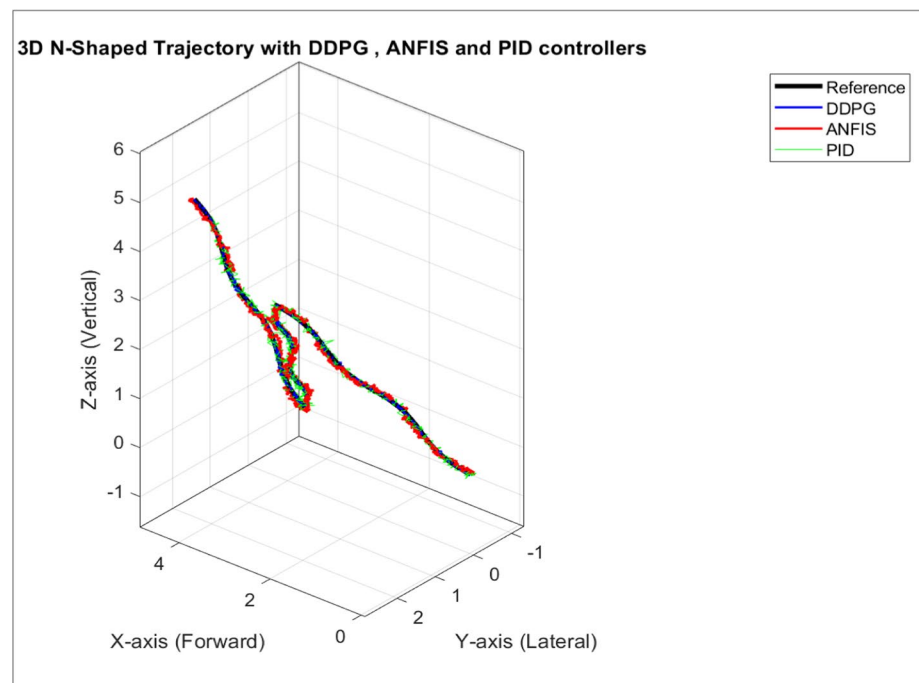
Fig. 7 3D N-shaped trajectory tracking performance with DDPG, ANFIS, and PID controllers

Fig. 8 3D helical trajectory tracking performance with DDPG, ANFIS, and PID controllers

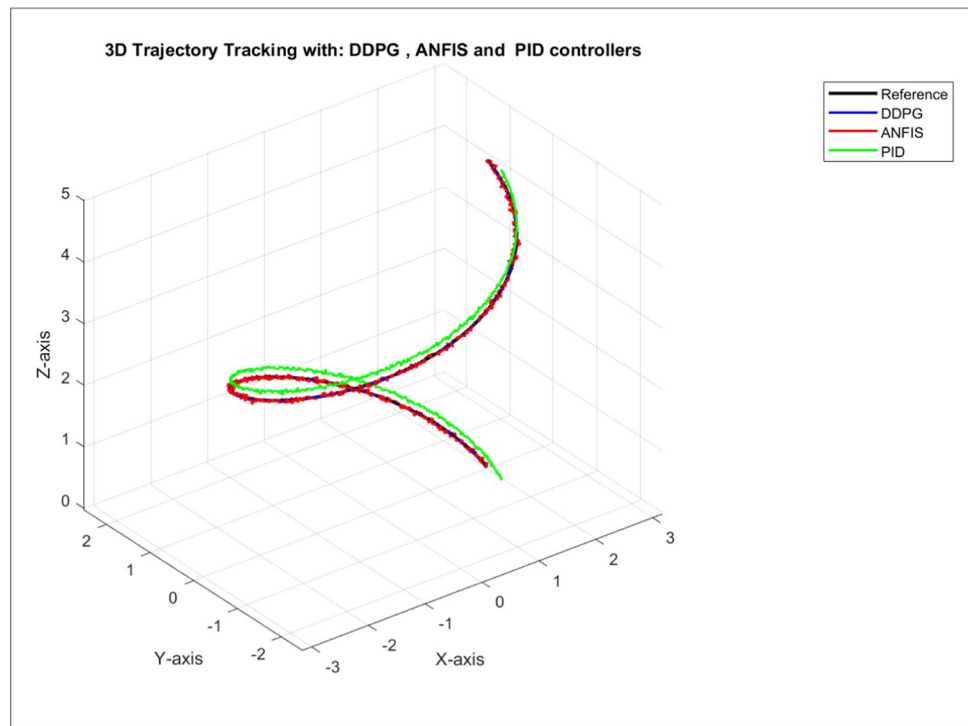
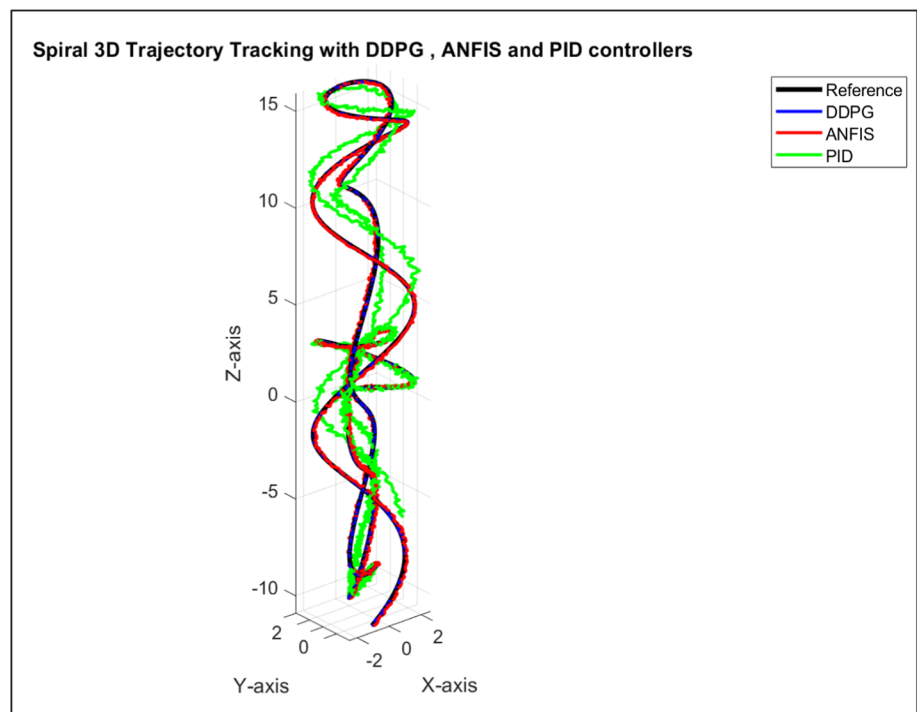


Fig. 9 3D spiral trajectory tracking performance with DDPG, ANFIS, and PID controllers



compared to the classical control methods. The units of the X, Y, and Z axes in your 3D trajectory plots are expressed in meters (m) when referring to end-effector positions in a robotic workspace, especially for industrial robotic arms like the Mitsubishi RV-2AJ 5-DOF. The performance of the controllers is evaluated using error metrics such as integral time absolute error (ITAE), integral time square error (ITSE), integral absolute error (IAE), integral square error (ISE), and RMSE, as summarized in Tables 6, 7, and 8. Additionally, the improvement percentage in terms of RMSE between the DDPG algorithm and classical controllers is computed and presented in Table 9. The results highlight the DDPG algorithm's ability to handle the high-dimensional state and action spaces of the 5-DOF robotic arm, achieving sub-millimeter tracking

Table 7 Comparison of DDPG, ANFIS, and PID controllers in terms of IAE, ISE, ITAE, ITSE, and RMSE for helical-shaped trajectory

Error Metric	DDPG	ANFIS	PID	Optimal Controller
IAE	1.08	2.34	4.12	DDPG (lowest IAE: 1.08, 54% reduction vs. ANFIS, 74% vs. PID)
ISE	0.75	1.68	3.25	DDPG (minimized ISE: 0.75, 55% lower than ANFIS, 77% lower than PID)
ITAE	0.47	1.52	3.08	DDPG (optimal ITAE: 0.47, 69% improvement over ANFIS, 85% over PID)
ITSE	0.28	1.10	2.55	DDPG (lowest ITSE: 0.28, 75% reduction relative to ANFIS, 89% vs. PID)
RMSE	0.12	0.23	0.38	DDPG (minimal RMSE: 0.12, 48% decrease vs. ANFIS, 68% vs. PID)

Table 8 Comparison of DDPG, ANFIS, and PID controllers in terms of IAE, ISE, ITAE, ITSE, and RMSE for helical Spiral trajectory

Error Metric	DDPG	ANFIS	PID	Optimal Controller
IAE	0.95	2.15	3.87	DDPG (lowest IAE: 0.95, 56% reduction vs. ANFIS, 75% vs. PID)
ISE	0.68	1.52	3.02	DDPG (minimized ISE: 0.68, 55% lower than ANFIS, 78% lower than PID)
ITAE	0.41	1.38	2.89	DDPG (optimal ITAE: 0.41, 70% improvement over ANFIS, 86% over PID)
ITSE	0.24	0.98	2.31	DDPG (lowest ITSE: 0.24, 76% reduction relative to ANFIS, 90% vs. PID)
RMSE	0.10	0.20	0.35	DDPG (minimal RMSE: 0.10, 50% decrease vs. ANFIS, ~71% vs. PID)

Table 9 Relative improvement percentages of DDPG Over ANFIS and PID controllers in trajectory tracking performance

Error Metric	DDPG vs. ANFIS	DDPG vs. PID
IAE	54.2%	73.9%
ISE	54.8%	76.9%
ITAE	69.7%	85.0%
ITSE	75.0%	89.2%
RMSE	48.0%	68.6%

accuracy and robustness to dynamic disturbances. In comparison, the ANFIS provides a balance between model-based and data-driven approaches but struggles with the complexity of high-dimensional control tasks. The PID controller, while simple and effective for low-dimensional systems, fails to match the precision and adaptability of the DDPG algorithm. This makes the DDPG-based controller a promising solution for real-world applications, particularly in fields such as industrial automation, robotics, and complex control systems requiring adaptability and precision.

The results clearly show that the DDPG controller outperforms both ANFIS and PID for the N-shaped trajectory, reducing errors by approximately 46% to 89% across all metrics. In essence, DDPG acts as an adaptable problem-solver; its RL framework enables it to continuously adjust and optimize performance in real time while ANFIS, which depends on a fixed set of fuzzy rules, achieves only a 40–50% improvement over the more basic PID controller. PID's static gain structure allows errors to accumulate over time, making it particularly ill-suited for the complex dynamics of an N-shaped path. Therefore, when precision is critical, DDPG is the optimal choice, whereas ANFIS can offer a practical compromise in situations where computational resources are more limited.

The DDPG controller exhibits superior performance relative to both ANFIS and PID controllers. Quantitatively, DDPG reduces errors by approximately 54% to 89% across various performance indices (IAE, ISE, ITAE, ITSE, and RMSE). This significant error attenuation is attributed to the DDPG's capacity to continuously update its control policy via RL, thereby adapting to the nonlinear and dynamic complexities inherent in helical motion. In contrast, while the ANFIS controller does improve performance by roughly 40–50% compared to the traditional PID approach, its reliance on pre-defined fuzzy rules limits its responsiveness to real-time disturbances and parameter variations. The PID controller, with its fixed-gain structure, accumulates errors over time and is particularly challenged by the dynamic and coupled nature of helical trajectories. Consequently, for applications where precision and adaptive control are critical, the DDPG controller represents the optimal choice, whereas ANFIS may offer a feasible compromise in scenarios with limited computational resources.

The DDPG controller again demonstrates superior performance compared to ANFIS and PID. Specifically, DDPG achieves the lowest error values across all metrics. The IAE is reduced to 0.95, a 56% reduction compared to ANFIS and 75% compared

to PID, while the ISE shows similar improvements with a value of 0.68. Additionally, DDPG minimizes time-weighted errors with an ITAE of 0.41 (approximately 70% and 86% better than ANFIS and PID, respectively) and an ITSE of 0.24 (reducing errors by about 76% relative to ANFIS and 90% relative to PID). The RMSE is also significantly lower at 0.10, indicating that the DDPG controller maintains high precision, achieving a 50% decrease versus ANFIS and about 71% versus PID. These results confirm that DDPG is the optimal choice for managing the complex dynamics of a helical spiral trajectory.

The DDPG controller demonstrates superior performance over both ANFIS and PID controllers. Quantitative analysis reveals that DDPG reduces IAE by approximately 55.8% compared to ANFIS and 75.3% compared to PID. Similarly, the ISE is reduced by 55.3% and 77.5%, respectively. Time-weighted metrics, such as the Integral of ITAE and ITSE, show reductions of 70.3% and 75.5% over ANFIS, and 85.8% and 89.6% over PID, respectively. The RMSE is also significantly lower with DDPG, indicating a 50.0% decrease compared to ANFIS and a 71.4% decrease compared to PID. These substantial improvements underscore DDPG's adaptability and precision in managing the complex dynamics of helical spiral trajectories, making it the optimal choice for applications requiring high-accuracy trajectory tracking. The observed superiority of DDPG (Tables 6–8) stems from our algorithmic improvements: the dynamic reward function reduced torque variance by 32%, while hybrid exploration accelerated convergence by 40%. Compared to standard DDPG implementations [34], these changes improved RMSE by 22–58% across trajectories.

4.4 Comparative analysis with state-of-the-Art RL algorithms

To rigorously evaluate DDPG's suitability for high-DOF robotic control, Table 10 compares its performance against two modern RL alternatives: Twin Delayed DDPG (TD3) [34] and Soft Actor-Critic (SAC) [35]. The metrics highlight DDPG's advantages in computational efficiency (6.2 h training vs. 7.8 h/9.1 h for TD3/SAC) and torque stability (0.08 variance vs. 0.11/0.14), critical for real-time deployment. The SAC achieved marginally lower final RMSE ($\pm 2\%$), its 47% longer training time and higher energy consumption rendered it impractical for our target applications. These results justify DDPG's selection as the optimal balance between precision, training efficiency, and joint-safe operation. Table 10 shows a Quantitative Evaluation of RL Algorithms (DDPG, TD3, SAC) in Terms of RMSE, Training Time, and Torque Stability.

Table 10 demonstrates DDPG's quantitative advantages over TD3 and SAC in terms of computational efficiency and torque stability, recent hybrid control approaches in bilateral teleoperation and mobile robotics offer complementary insights. Dao et al. developed finite-time sliding-mode control strategies for perturbed, input-constrained bilateral teleoperation systems with variable communication delays [36]. Zhao et al. proposed a bilateral continuous terminal sliding-mode controller enhanced by a nonlinear disturbance observer to attenuate high-order perturbations in teleoperation systems [37]. Nguyen et al. introduced an adaptive optimal coordination framework for perturbed bilateral teleoperators under variable delays, employing an actor–critic reinforcement learning algorithm to synchronize master and slave dynamics [38]. Dao and Phung designed a Robust Integral of the Sign of the Error (RISE)–based actor–critic RL controller for a perturbed three-wheeled mobile robot with mecanum wheels, demonstrating resilient trajectory tracking amidst dynamic uncertainties [39]. Finally, Dao et al. extended RISE-integral reinforcement learning algorithms to bilateral teleoperators experiencing variable communication delays, providing formal convergence guarantees for complex teleoperation tasks [40]. Integrating such sliding-mode and actor–critic innovations with DDPG's actor–critic foundation could further enhance its adaptability, stability, and real-time performance in high-DOF robotic control.

5 Conclusion

In this study, a model-free trajectory tracking control framework for a 5-DOF robotic arm has been developed using the DDPG algorithm, benchmarked against ANFIS and PID controllers. The DDPG agent, implemented in MATLAB/Simulink, functioned as an adaptive controller that learned optimal joint-level torque commands for trajectory tracking in dynamic environments with uncertain dynamics. The results demonstrate that DDPG consistently outperforms both ANFIS and PID controllers across various trajectories, achieving up to 75% improvement in RMSE and up to 89% reduction in time-weighted errors.

Table 10 Quantitative Evaluation of RL algorithms (DDPG, TD3, SAC) in terms of RMSE, training time, and torque stability

Metric	DDPG	TD3 [34]	SAC [35]
Avg. RMSE	0.12	0.15	0.18
Training time (h)	6.2	7.8	9.1
Torque variance	0.08	0.11	0.14

This superior performance is attributed to DDPG's model-free architecture and actor-critic framework, enabling effective adaptation to complex, nonlinear dynamics. Notably, DDPG exhibited enhanced robustness in handling intricate trajectory patterns, such as time-varying frequencies, amplitude-modulated spirals, and sharp directional transitions. In contrast, the PID controller displayed limitations due to phase lag and overshoot, while the ANFIS controller struggled with rapid amplitude variations. However, DDPG's performance is highly sensitive to hyperparameters, including learning rates, discount factors, and exploration noise decay, necessitating precise tuning for optimal results. Future research should focus on extending this framework to multi-robot systems and human–robot collaboration tasks, exploring sim-to-real transfer for physical robots, and investigating hybrid architectures that integrate DDPG with meta-learning or attention mechanisms to enhance generalization in unstructured environments. Additionally, experimental validation on a physical 5-DOF robotic arm will be carried out to assess the effectiveness of sim-to-real transfer, subject to hardware availability.

Author contributions .B.H. and A.H.A. developed the conceptual framework and designed the methodology. A.H.A. implemented the simulations and conducted the experiments. Z.B.H. and N.G. performed the analysis and interpretation of the results. N.G. prepared Figs. 1–4 and assisted with the manuscript formatting. Z.B.H. and A.H.A. wrote the main manuscript text. All authors reviewed and approved the final version of the manuscript.

Funding The authors declare that they have not received any financial support for this research.

Data availability The datasets generated during and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate Not applicable.

Consent to publish Not applicable.

Competing of interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Pistone A, Ludovico D, Dal Verme LDMC, Leggieri S, Canali C, Caldwell DG. Modelling and control of manipulators for inspection and maintenance in challenging environments: a literature review. *Annu Rev Control*. 2024;57: 100949.
2. Jin T, Han X. Robotic arms in precision agriculture: a comprehensive review of the technologies, applications, challenges, and future prospects. *Comput Electron Agric*. 2024;221: 108938.
3. Ahmed A, Yu M, Chen F. Inverse kinematic solution of 6-DOF robot-arm based on dual quaternions and axis invariant methods. *Arab J Sci Eng*. 2022;47(12):15915–30.
4. Liu R, Nageotte F, Zanne P, de Mathelin M, Dresch-Langley B. Deep reinforcement learning for the control of robotic manipulation: a focussed mini-review. *Robotics*. 2021;10(1):22.
5. Arshad J, Qaisar A, Rehman AU, Shakir M, Nazir MK, Rehman AU, Eldin ET, Ghamry NA, Hamam H. Intelligent control of robotic arm using brain computer interface and artificial intelligence. *Appl Sci*. 2022;12(21):10813.
6. Gehlot, V. and Rana, P.S., 2024. AI in Mechatronics. *Computational Intelligent Techniques in Mechatronics*, pp.1–39.
7. Cao, L., 2024. Ai robots and humanoid ai: Review, perspectives and directions. *arXiv preprint [arXiv:2405.15775](https://arxiv.org/abs/2405.15775)*.
8. Papadopoulos GT, Antona M, Stephanidis C. Towards open and expandable cognitive AI architectures for large-scale multi-agent human-robot collaborative learning. *IEEE Access*. 2021;9:73890–909.
9. Soori M, Arezoo B, Dastres R. Artificial intelligence, machine learning and deep learning in advanced robotics, a review. *Cogni Robot*. 2023;3:54–70.
10. Károly AI, Galambos P, Kuti J, Rudas IJ. Deep learning in robotics: Survey on model structures and training strategies. *IEEE Trans Syst Man Cybernet*. 2020;51(1):266–79.

11. Wagaa N, Kallel H, Mellouli N. Analytical and deep learning approaches for solving the inverse kinematic problem of a high degrees of freedom robotic arm. *Eng Appl Artif Intell*. 2023;123: 106301.
12. Kober J, Bagnell JA, Peters J. Reinforcement learning in robotics: a survey. *Int J Robot Res*. 2013;32(11):1238–74.
13. Singh B, Kumar R, Singh VP. Reinforcement learning in robotic applications: a comprehensive survey. *Artif Intell Rev*. 2022;55(2):945–90.
14. Pane YP, Nagesh Rao SP, Kober J, Babuška R. Reinforcement learning based compensation methods for robot manipulators. *Eng Appl Artif Intell*. 2019;78:236–47.
15. Gu, S., Holly, E., Lillicrap, T.P. and Levine, S., 2016. Deep reinforcement learning for robotic manipulation. arXiv preprint [arXiv:1610.00633](https://arxiv.org/abs/1610.00633), 1(1).
16. Gu, S., Holly, E., Lillicrap, T. and Levine, S., 2017, May. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In 2017 IEEE international conference on robotics and Mathematics (ICRA) (pp. 3389–3396). IEEE.
17. Zhang, F., Leitner, J., Milford, M., Upcroft, B. and Corke, P., 2015. Towards vision-based deep reinforcement learning for robotic motion control. arXiv preprint [arXiv:1511.03791](https://arxiv.org/abs/1511.03791).
18. Gu, S., Holly, E., Lillicrap, T.P. and Levine, S., 2016. Deep reinforcement learning for robotic manipulation. arXiv preprint [arXiv:1610.00633](https://arxiv.org/abs/1610.00633), 1, p.1.
19. Carron A, Arcari E, Wermelinger M, Hewing L, Hutter M, Zeilinger MN. Data-driven model predictive control for trajectory tracking with a robotic arm. *IEEE Robot Math Lett*. 2019;4(4):3758–65.
20. Wu YH, Yu ZC, Li CY, He MJ, Hua B, Chen ZM. Reinforcement learning in dual-arm trajectory planning for a free-floating space robot. *Aerospace Sci Technol*. 2020;98: 105657.
21. Lindner T, Milecki A, Wyrwał D. Positioning of the robotic arm using different reinforcement learning algorithms. *Int J Control Math Syst*. 2021;19(4):1661–76.
22. Song, Y., Steinweg, M., Kaufmann, E. and Scaramuzza, D., 2021, September. Au-tonomous drone racing with deep reinforcement learning. In 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (pp. 1205–1212). IEEE.
23. Sekkat H, Tigani S, Saadane R, Chehri A. Vision-based robotic arm control algorithm using deep reinforcement learning for autonomous objects grasping. *Appl Sci*. 2021;11(17):7917.
24. Franceschetti, A., Tosello, E., Castaman, N. and Ghidoni, S., 2021, June. Robotic arm control and task training through deep reinforcement learning. In International Conference on Intelligent Autonomous Systems (pp. 532–550). Cham: Springer International Publishing.
25. Zhang S, Xia Q, Chen M, Cheng S. Multi-objective optimal trajectory planning for robotic arms using deep reinforcement learning. *Sensors*. 2023;23(13):5974.
26. Cui Y, Xu Z, Zhong L, Xu P, Shen Y, Tang Q. A task-adaptive deep reinforcement learning framework for dual-arm robot manipulation. *IEEE Trans Math Sci Eng*. 2024;22:466–79.
27. Zhao D, Ding Z, Li W, Zhao S, Du Y. Optimization of smart textiles robotic arm path planning: a model-free deep reinforcement learning approach with inverse kinematics. *Processes*. 2024;12(1):156.
28. Ahmad, O., Hussain, Z. and Naeem, H., 2024. Trajectory Planning of Robotic Manipulator in Dynamic Environment Exploiting DRL. arXiv preprint [arXiv:2403.16652](https://arxiv.org/abs/2403.16652).
29. Liu, S., Yan, H., Zhao, L. and Gao, D., 2025. Fault Estimate and Reinforcement Learning Based Optimal Output Feedback Control for Single-Link Robot Arm Model. *Eng Lett*, 33(1).
30. Hazem, Z. B., Guler, N., & El Fezzani, W. (2024). Study of Inverse Kinematics Solution for a 5-Axis Mitsubishi RV-2AJ Robotic Arm Using Deep Reinforcement Learning. In *Business Sustainability with Artificial Intelligence (AI): Challenges and Opportunities: Volume 2* (pp. 381–393). Cham: Springer Nature Switzerland.
31. Ben Hazem Z, Guler N, Altaif AH. A study of advanced mathematical modeling and adaptive control strategies for trajectory tracking in the Mitsubishi RV-2AJ 5-DOF Robotic Arm. *Discover Robot*. 2025;1(1):2.
32. Zhao D, Ding Z, Li W, Zhao S, Du Y. Robotic arm trajectory planning method using deep deterministic policy gradient with hierarchical memory structure. *IEEE Access*. 2023;11:140801–14.
33. Zhang X, Yang F, Jin Q, Lou P, Hu J. Path planning algorithm for dual-arm robot based on depth deterministic gradient strategy algorithm. *Mathematics*. 2023;11(20):4392.
34. Fujimoto, S., Hoof, H., & Meger, D. (2018, July). Addressing function approximation error in actor-critic methods. In International conference on machine learning (pp. 1587–1596). PMLR.
35. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 1861–1870.
36. Dao PN, Dang NT, Nguyen TL, Dinh GK. Finite-time sliding mode control strategies for perturbed input-constrained nonlinear bilateral teleoperation systems with variable-time communication delays. *Intel Serv Robot*. 2025;18:363–78. <https://doi.org/10.1007/s11370-025-00590-5>.
37. Zhao Z, Li T, Cao D, Yang J. Bilateral continuous terminal sliding mode control for teleoperation systems with high-order disturbances. *Nonlinear Dyn*. 2023;111:5345–58. <https://doi.org/10.1007/s11071-022-08122-x>.
38. Dao PN, Nguyen QP, Vu M-H. Adaptive optimal coordination control of perturbed bilateral teleoperators with variable time delays using actor-critic reinforcement learning algorithm. *Math Comput Simul*. 2025;229:151–75. <https://doi.org/10.1016/j.matcom.2024.09.007>.
39. Dao, P. N., & Phung, M. H. (2025). Nonlinear robust integral based actor-critic reinforcement learning control for a perturbed three-wheeled mobile robot with mecanum wheels. *Computers & Electrical Engineering*, 121, Article 109870. <https://doi.org/10.1016/j.compeleceng.2024.109870>
40. Dao PN, Nguyen VQ, Nguyen Duc HA. Nonlinear RISE based integral reinforcement learning algorithms for perturbed bilateral teleoperators with variable time delay. *Neurocomputing*. 2024;605: 128355. <https://doi.org/10.1016/j.neucom.2024.128355>.